

Практика 14. Рекурсивные типы

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2024

В предыдущих сериях

В предыдущих сериях

- ▶ Комбинатор неподвижной точки
 - ▶ Заводим магическую функцию `fix` (e.g. комбинатор **Y** внутри)
 - ▶ Всю рекурсию можно выразить через неё
- `todo` Как написать факториал через `fix`?

В предыдущих сериях

- ▶ Комбинатор неподвижной точки

- ▶ Заводим магическую функцию `fix` (e.g. комбинатор **Y** внутри)

- ▶ Всю рекурсию можно выразить через неё

`todo` Как написать факториал через `fix`?

⚡ `fac = fix \rec n -> if n <= 1 then 1 else n * rec (n - 1)`

В предыдущих сериях

- ▶ Комбинатор неподвижной точки
 - ▶ Заводим магическую функцию `fix` (e.g. комбинатор **Y** внутри)
 - ▶ Всю рекурсию можно выразить через неё
- `todo` Как написать факториал через `fix`?
 - ⚡ `fac = fix \rec n -> if n <= 1 then 1 else n * rec (n - 1)`
- ▶ Алгебраические типы данных
- ▶ Рекурсивные алгебраические типы данных
 - `data List a = Nil | Cons a (List a)`

В предыдущих сериях

- ▶ Комбинатор неподвижной точки
 - ▶ Заводим магическую функцию `fix` (e.g. комбинатор **Y** внутри)
 - ▶ Всю рекурсию можно выразить через неё
- todo Как написать факториал через `fix`?
 - ⚡ `fac = fix \rec n -> if n <= 1 then 1 else n * rec (n - 1)`
- ▶ Алгебраические типы данных
- ▶ Рекурсивные алгебраические типы данных
 - `data List a = Nil | Cons a (List a)`
- + Структурная запись алгебраических типов
- + μ -нотация для кодирования структурных рекурсивных типов
- + Кодирование рекурсивных типов в Haskell через неподвижную точку
- + Катаморфизмы, анаморфизмы, гилеморфизмы

1. Обобщённые свёртки и развёртки

2. Эпилог

Содержание

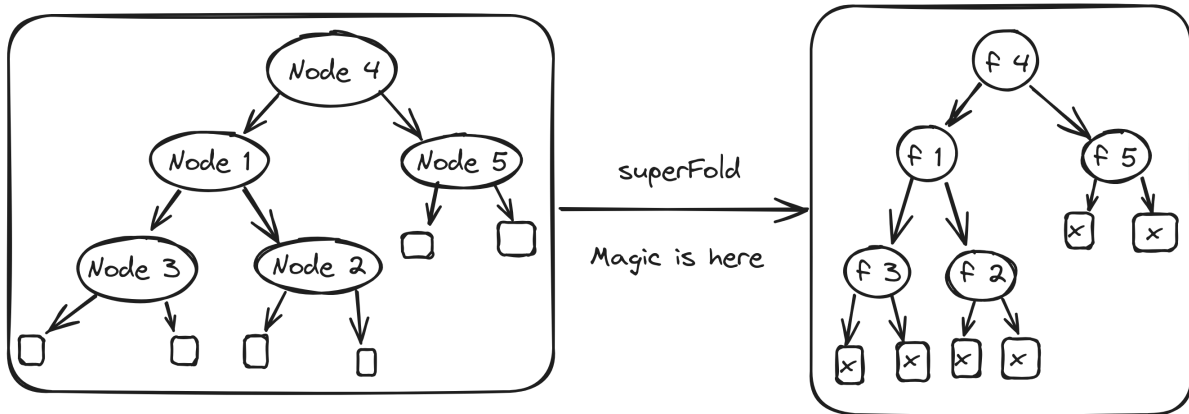
1. Обобщённые свёртки и развёртки

2. Эпилог

Обобщённые свёртки и развёртки

- ▶ Хотим обобщить свёртки и развёртки со списком на произвольные типы
- ▶ Все конструкторы заменяем на соответствующие функции

```
data Tree a = Leaf | Node (Tree a) a (Tree a)
```



Форма дерева — отдельно!

- ▶ Можем взять произвольную рекурсивную структуру данных

```
data List a = Nil | Cons a (List a)
```

- ▶ И разделить на комбинатор рекурсии и нерекурсивный тип, задающий форму

```
data Shape a rec = FNil | FCons a rec
```

```
data Fix f = In { out :: f (Fix f) }
```

```
type FList a = Fix (Shape a)
```

todo Задайте список [1, 2] только через **Shape**, какой будет тип?

todo Задайте список [1, 2], используя **Fix**

todo Покажите изоморфизм между **List** и **FList**

Форма дерева — отдельно!

- ▶ Можем взять произвольную рекурсивную структуру данных

```
data List a = Nil | Cons a (List a)
```

- ▶ И разделить на комбинатор рекурсии и нерекурсивный тип, задающий форму

```
data Shape a rec = FNil | FCons a rec
```

```
data Fix f = In { out :: f (Fix f) }
```

```
type FList a = Fix (Shape a)
```

todo Задайте список [1, 2] только через **Shape**, какой будет тип?

todo Задайте список [1, 2], используя **Fix**

todo Покажите изоморфизм между **List** и **FList**



```
FCons 1 (FCons 2 SNil) :: Shape Int (Shape Int (Shape a))
```

Форма дерева — отдельно!

- ▶ Можем взять произвольную рекурсивную структуру данных

```
data List a = Nil | Cons a (List a)
```

- ▶ И разделить на комбинатор рекурсии и нерекурсивный тип, задающий форму

```
data Shape a rec = FNil | FCons a rec
```

```
data Fix f = In { out :: f (Fix f) }
```

```
type FList a = Fix (Shape a)
```

todo Задайте список [1, 2] только через **Shape**, какой будет тип?

todo Задайте список [1, 2], используя **Fix**

todo Покажите изоморфизм между **List** и **FList**

```
⚡ FCons 1 (FCons 2 SNil) :: Shape Int (Shape Int (Shape a))
```

```
⚡ In $ FCons 1 $ In $ FCons 2 $ In FNil :: FList Int
```

Форма дерева — отдельно!

- ▶ Можем взять произвольную рекурсивную структуру данных

```
data List a = Nil | Cons a (List a)
```

- ▶ И разделить на комбинатор рекурсии и нерекурсивный тип, задающий форму

```
data Shape a rec = FNil | FCons a rec
```

```
data Fix f = In { out :: f (Fix f) }
```

```
type FList a = Fix (Shape a)
```

todo Задайте список [1, 2] только через **Shape**, какой будет тип?

todo Задайте список [1, 2], используя **Fix**

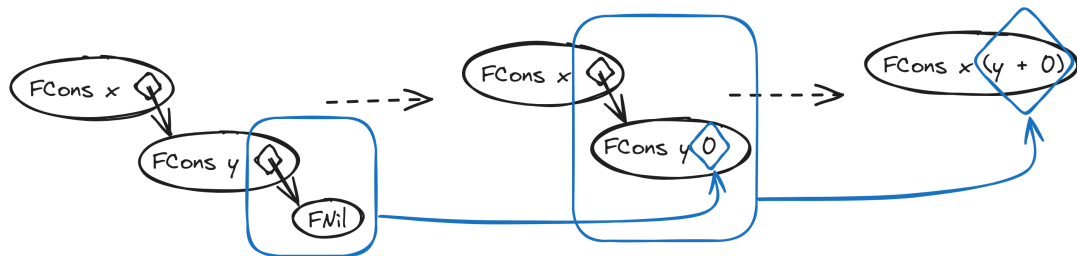
todo Покажите изоморфизм между **List** и **FList**

```
⚡ FCons 1 (FCons 2 SNil) :: Shape Int (Shape Int (Shape a))
```

```
⚡ In $ FCons 1 $ In $ FCons 2 $ In FNil :: FList Int
```

```
⚡ to xs = case xs of Nil -> In FNil; Cons x xs' -> In (FCons x (to xs'))
from xs = case out xs of FNil -> Nil; FCons x xs' -> Cons x (from xs')
```

Катаморфизм внутри



- Будем подменять ссылки на поддеревья на частичные результаты свёртки

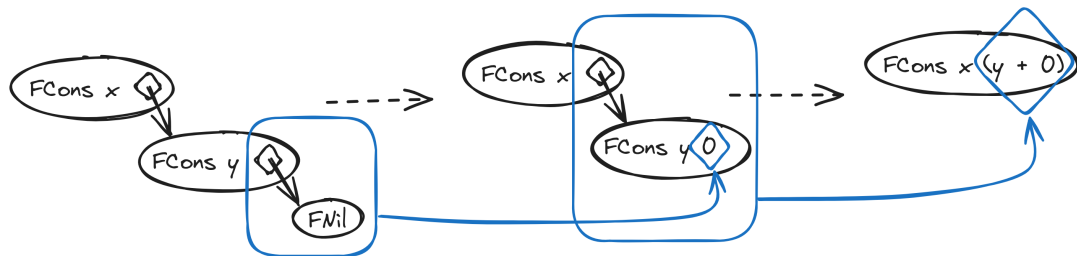
```
instance Functor (Shape a) where
```

```
  fmap :: (rec -> value) -> L a rec -> L a value
```

```
  fmap f = \case FNil -> FNil; FCons x rec -> FCons x (f rec)
```

todo Напишите функцию `cata`

Катаморфизм внутри



- Будем подменять ссылки на поддеревья на частичные результаты свёртки

```
instance Functor (Shape a) where
```

```
  fmap :: (rec -> value) -> L a rec -> L a value
```

```
  fmap f = \case FNil -> FNil; FCons x rec -> FCons x (f rec)
```

todo Напишите функцию `cata`



```
cata :: Functor shape => (shape b -> b) -> Fix shape -> b
```

```
cata phi = phi . fmap (cata phi) . out
```

- Анаморфизм аналогично, только храним зёрна для разворачивания поддеревьев

Использование катаморфизмов

```
cata :: Functor shape => (shape b -> b) -> Fix shape -> b
```

todo Запишите через неподвижную точку

```
data Expr = Zero' | Succ' Expr | Plus' Expr Expr
```

todo Сверните FExpr в число

todo Сверните FExpr в строку

todo Почему $f \ a \rightarrow a$ называют f -алгеброй?

Использование катаморфизмов

```
cata :: Functor shape => (shape b -> b) -> Fix shape -> b
```

todo Запишите через неподвижную точку

```
data Expr = Zero' | Succ' Expr | Plus' Expr Expr
```

todo Сверните FExpr в число

todo Сверните FExpr в строку

todo Почему $f \ a \rightarrow a$ называют f -алгеброй?



```
data E rec = Zero | Succ rec | Plus rec rec deriving Functor
type FExpr = Fix E
```

Использование катаморфизмов

```
cata :: Functor shape => (shape b -> b) -> Fix shape -> b
```

todo Запишите через неподвижную точку

```
data Expr = Zero' | Succ' Expr | Plus' Expr Expr
```

todo Сверните FExpr в число

todo Сверните FExpr в строку

todo Почему $f \ a \rightarrow a$ называют f -алгеброй?

```
⚡ data E rec = Zero | Succ rec | Plus rec rec deriving Functor
   type FExpr = Fix E
```

```
⚡ phiInt :: E Int -> Int; phiInt = \case Zero -> 0; Succ x -> x + 1; ...
   evalExpr :: FExpr -> Int; evalExpr = cata phiInt
```

Использование катаморфизмов

```
cata :: Functor shape => (shape b -> b) -> Fix shape -> b
```

todo Запишите через неподвижную точку

```
data Expr = Zero' | Succ' Expr | Plus' Expr Expr
```

todo Сверните FExpr в число

todo Сверните FExpr в строку

todo Почему $f \ a \rightarrow a$ называют f -алгеброй?

```
data E rec = Zero | Succ rec | Plus rec rec deriving Functor
type FExpr = Fix E
```

```
phiInt :: E Int -> Int; phiInt = \case Zero -> 0; Succ x -> x + 1; ...
evalExpr :: FExpr -> Int; evalExpr = cata phiInt
```

```
phiShow :: E String -> String;
phiShow = \case
  Zero -> "0"; Succ x -> show x ++ "+1"
  Plus l r -> '(' : show l ++ '+' : show r ++ ")"
```

Magic (*)

- ▶ Катаморфизмы показывают наличие следующего изоморфизма

```
cata :: Functor f => Fix f -> (forall a. (f a -> a) -> a)
uncata :: (forall a. (f a -> a) -> a) -> Fix f -- = ($ In)
```

⇒ Любую рекурсивную структуру данных можно закодировать просто как функцию

- ▶ Возьмём форму списка и преобразуем

```
data LShape e rec = FNil | FCons e rec
```

⇒ Экспонента суммы равна произведению экспонент

$$a^{1+e \times a} \simeq a^1 \times a^{e \times a} \simeq a \times a^{a^e}$$

todo Что-нибудь напоминает?

Magic (*)

- ▶ Катаморфизмы показывают наличие следующего изоморфизма

```
cata :: Functor f => Fix f -> (forall a. (f a -> a) -> a)
uncata :: (forall a. (f a -> a) -> a) -> Fix f -- = ($ In)
```

⇒ Любую рекурсивную структуру данных можно закодировать просто как функцию

- ▶ Возьмём форму списка и преобразуем

```
data LShape e rec = FNil | FCons e rec
```

⇒ Экспонента суммы равна произведению экспонент

$$a^{1+e \times a} \simeq a^1 \times a^{e \times a} \simeq a \times a^{a^e}$$

todo Что-нибудь напоминает?



Да это же наше “кодирование наоборот” из лямбда-исчисления!

```
[e] ≃ Fix (LShape e) ≃ (forall a. a -> (e -> a -> a) -> a)
```

Содержание

1. Обобщённые свёртки и развёртки

2. Эпилог

Зачёты и формальности

Зачёт

- ▶ Дорешивание разобранных задач за полцены
- ▶ Дорешивание неразобранных за полный балл¹
- ▶ Если не хватит (что вряд ли) можно приходить за новыми задачами

¹Не все указаны в табличке, приходите и напоминайте, что не разбирали.

Экзамен

- ▶ Определяет оценку за курс, для допуска нужен зачёт
- ▶ Можно договариваться с ДН и с нами про консультации
- ▶ Команде курса показывает, насколько у них что-то (не)получилось
- ▶ Вам — насколько вы усвоили материал + возможность привести всё в порядок
- ▶ Кураторам — сколько вам плотить деняк

Понравился Haskell, что дальше?

- ▶ Материалы со слайдов
- ▶ Виталий Брагилевский “Haskell in depth”
- ▶ Sandy Maguire “Thinking with types: typelevel programming in Haskell”
- ▶ Sandy Maguire “Algebra Driven Design”
- ▶ Simon Marlow “Concurrent programming in Haskell”
- ▶ Много пейперов!

Понравился Haskell, что дальше?

- ▶ Материалы со слайдов
- ▶ Виталий Брагилевский “Haskell in depth”
- ▶ Sandy Maguire “Thinking with types: typelevel programming in Haskell”
- ▶ Sandy Maguire “Algebra Driven Design”
- ▶ Simon Marlow “Concurrent programming in Haskell”
- ▶ Много пейперов!
- ▶ Можно поискать стажировку и работу: tweag.io, [serokell](https://serokell.com), [biocad](https://biocad.ru), [Сириус.Курсы](https://Сириус.Курсы.рф)...

Мне понравился курс, что со мной не так?

Главное не паникуйте, вдох-выдох. Это почти нормально. Возможно, вы PL'щик и вам стоит пойти на трек языков программирования.

Что там изучать? • Типы в языках программирования • Компиляторы •
Программирование с зависимыми типами • Гомотопическая теория типов (HOTT)
• Виртуальные машины • Метапрограммирование • Формальная верификация •
Статический анализ • Семантика ЯПов • Теория категорий • ...

Временами это удовольствие совсем на любителя. Но если ваше сердечко лежит к языкам, очень вероятно, что вы найдете в них местечко себе по вкусу.

Задача

Задача

Вы ходили на пары
Задавали вопросы
Делали домашки
Запрашивали помощь
И помогали друг другу

Задача

Вы ходили на пары
Задавали вопросы
Делали домашки
Запрашивали помощь
И помогали друг другу

А значит, вы большие молодцы ♥

